



Obsidian

AUDITS

Bounce Tech Security Review

Auditors

Oxjuaan
OxSpearmin

April 28, 2026

Introduction

Obsidian Audits

Obsidian audits is a team of top-ranked security researchers, with a publicly proven track record, specialising in DeFi protocols across EVM chains and Solana.

The team has achieved 10+ top-2 placements in audit competitions, placing 1st in competitions for Wormhole, Pump.fun, Yearn Finance, and many more.

Find out more: obsidianaudits.com

Audit Overview

Bounce Tech is a leveraged token protocol, built on HyperEVM.

Bounce Tech engaged Obsidian Audits to perform a security review of a smart contract update to support creating leveraged tokens for HIP-3 perps. The review was conducted from April 27, 2026 to April 28, 2026.

Scope

Files in scope

Repo	Files in scope
<code>bounce-contracts</code> Commit hash: fd9245b1b1f4df33332541 639d9750d012f6df25	PR #204, #205, #206

Summary of Findings

Severity Breakdown

A total of 1 issues were identified and categorized based on severity:

- 1 Low severity

Findings Overview

ID	Title	Severity	Status
L-01	Separate HIP-3 creation path bypasses on-chain validation	Low	Fixed

Severity Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users. Alternatively, breaking a core aspect of the protocol's intended functionality
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - the attack/vulnerability requires minimal or no preconditions, but there is limited or no incentive to exploit it in practice
- **Low** - requires highly unlikely precondition states, or requires a significant attacker capital with little or no incentive.

Findings

[L-01] Separate HIP-3 creation path bypasses on-chain validation

Description

`Factory` exposes two creation paths: `createLt` for validator perps and `createHip3Lt` for HIP-3 perps (asset ids $\geq 100_000$). The split was introduced in [commit 11c059a](#) with the comment: "the legacy `perpAssetInfo` precompile (0x80a) does not understand [HIP-3 ids], so any HIP-3 market metadata must be supplied by the caller instead."

The precompile is keyed by perp index rather than `assetId`, and does return valid metadata for HIP-3 markets when called with the perp index. As written, `createHip3Lt` instead takes `targetAsset_` from the owner without verification and does not call `_validateMaxLeverage`, and `redeployLt` skips the same check via its `marketId_ < _HIP_3_BASE` guard.

The `marketId_` stored on each leveraged token is the Hyperliquid `assetId`, which for HIP-3 markets is encoded as $100_000 + \text{perpDexIndex} * 10_000 + \text{indexInMeta}$. The `assetId` is the value used to place orders. Read precompiles such as `perpAssetInfo`, `markPx`, and `oraclePx` instead expect the perp index, which can be calculated from the `assetId` as follows:

```
perpIndex = assetId >= 100_000 ? assetId - 100_000 : assetId.
```

For example, `xyz:TSLA` has `assetId` `110001` and perp index `10001` for precompile reads, which can be verified against this [L1Read contract](#).

Recommendation

Consider collapsing both paths into a single `createLt` by converting `marketId_` to a perp index before calling `PrecompileLib.perpAssetInfo`, then use the returned `coin` as `targetAsset_` and run `_validateMaxLeverage`.

Apply the same conversion in `redeployLt` so its existing `marketId_ < _HIP_3_BASE` guard around `_validateMaxLeverage` can be dropped.

```

function createLt(uint32 marketId_, ...) external onlyOwner
returns (address) {
-   if (marketId_ >= _HIP_3_BASE) revert InvalidMarketId();
    ...
-   PrecompileLib.PerpAssetInfo memory perpAssetInfo_ =
PrecompileLib.perpAssetInfo(marketId_);
+   uint32 perpIndex_ = marketId_ >= _HIP_3_BASE ? marketId_
-   _HIP_3_BASE : marketId_;
+   PrecompileLib.PerpAssetInfo memory perpAssetInfo_ =
PrecompileLib.perpAssetInfo(perpIndex_);
    ...
}

function redeployLt(address ltAddress_) external onlyOwner {
    ...
-   if (marketId_ < _HIP_3_BASE) {
-       _validateMaxLeverage(PrecompileLib.perpAssetInfo(marketId_),
targetLeverage_);
-   }
+   uint32 perpIndex_ = marketId_ >= _HIP_3_BASE ? marketId_
-   _HIP_3_BASE : marketId_;
+   _validateMaxLeverage(PrecompileLib.perpAssetInfo(perpIndex_),
targetLeverage_);
    ...
}

```

Also, consider changing the `marketId` name to `assetId` since that is what the id represents, rather than the perp market ID which is different.

Remediation: Fixed in commit [274b542](#)